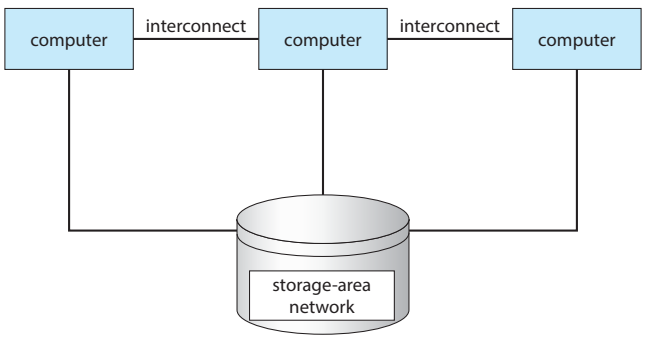
می‌توانند اجرای آن را بر عهده بگیرند. در یک خوشه‌ی پایگاه‌داده، چندین میزبان می‌توانند پایگاه‌داده را به اشتراک بگیرند تا کارایی و قابلیت اطمینان، به صورت قابل توجهی افزایش یابد. شکل ۱-۱۱، ساختار کلی سیستم خوشه‌ای را به تصویر می‌کشد.

۱-۴ عملیات سیستم‌عامل

اکنون که اطلاعات پایه‌ای مربوط به سازمان و معماری سیستم کامپیوتری را بحث کردیم، آماده‌ایم تا در رابطه با سیستم‌عامل‌ها صحبت کنیم. سیستم‌عامل بستری را فراهم می‌کند که در آن برنامه‌ها اجرا می‌شوند. سیستم‌عامل‌ها از نظر ساخت داخلی، تفاوت زیادی با همدیگر دارند، چرا که آنها با استفاده از خطوط کد متفاوت بسیاری سازمان‌دهی می‌شوند. با این وجود، مشترکات زیادی هستند که در این بخش به آنها می‌پردازیم.

یک کامپیوتر برای شروع اجرا—برای نمونه، موقعی که روشن می‌شود یا دومرتبه راه‌اندازی می‌گردد—نیاز دارد تا یک برنامه‌ی آغازین برای اجرا داشته باشد. چنان‌چه پیشتر عنوان شد، این برنامه‌ی آغازین، یا برنامه‌ی بالاآورنده، بایستی ساده باشد. نوعاً، در میان سخت‌افزار کامپیوتر در سفت‌افزار ذخیره می‌شوند. آن تمامی جوانب سیستم را از ثبات‌های CPU گرفته تا کنترلر دستگاه‌ها تا محتویات حافظه مقداردهی اولیه می‌نماید. برنامه‌ی بالاآورنده بایستی بداند که چگونه سیستم‌عامل را بارگذاری نماید و چگونه اجرای آن سیستم را شروع کند. برای انجام این منظور، برنامه‌ی بالاآورنده بایستی هسته‌ی[1] سیستم‌عامل را پیدا کرده و آن را داخل حافظه بارگذاری نماید.



شکل ۱-۱۱ ساختار کلی سیستم خوشه‌ای.

همین که هسته بار شد و اجرایش را شروع کرد، می‌تواند تدارک سرویس به سیستم و کاربران آن را آغاز کند. برخی سرویس‌ها در بیرون از هسته توسط برنامه‌های سیستمی فراهم می‌شوند که در زمان راه‌اندازی به‌داخل حافظه بارگذاری می‌شوند تا **گماشته‌های سیستمی**[2] باشند که در تمامی زمان‌هایی که هسته اجرا دارد اجرا داشته باشند. در لینوکس، اولین برنامه‌ی

1. Kernel
2. System daemons

سیستمی، «systemd» می‌باشد که بسیاری از گماشتگان دیگر را نیز شروع می‌نماید. همین که این مرحله تکمیل شد، سیستم به‌صورت کامل راه‌اندازی شده و سیستم به‌انتظار رویدادی می‌ماند که اتفاق بیفتد.

اگر هیچ پردازه‌ای برای اجرا وجود نداشته باشد، هیچ دستگاه I/O برای سرویس نباشد، و هیچ کاربری نباشد که بایستی پاسخ داده شود، سیستم‌عامل، در انتظار رخداد چیزی آرام خواهد نشست. رویدادها تقریباً همیشه توسط رخداد یک وقفه علامت داده می‌شوند. شکل دیگری از وقفه **تله**[1] (یا یک **استثناء**) می‌باشد که یک وقفه‌ی تولید شده‌ی نرم‌افزاری است که به‌واسطه‌ی یک خطا (برای مثال، تقسیم بر صفر یا دسترسی حافظه‌ی نامعتبر) و یا توسط یک درخواست به‌خصوص از جانب یک برنامه‌ی کاربر برای اجرای یک سرویس سیستم‌عامل با یک عملیات خاص به‌نام **فراخوانی سیستمی**[2] اجرا می‌شود.

HADOOP

Hadoop یک چارچوب نرم‌افزاری متن‌باز است که برای پردازش توزیع‌شده‌ی مجموعه‌های داده‌ای بزرگ (تحت عنوان **کلان داده‌ها**[3]) در یک سیستم خوشه‌ای، شامل اجزای سخت‌افزاری ساده‌ی کم هزینه استفاده می‌شود. Hadoop طراحی شده است تا از یک سیستم منفرد تا یک خوشه که شامل هزاران گره محاسباتی است، گسترش داشته باشد. وظایف به یک گره خوشه گمارده می‌شوند و Hadoop ارتباطات میان گره‌ها را جفت‌وجور می‌کند تا محاسبات موازی را مدیریت کرده و نتایج را جمع‌بندی کند. Hadoop خرابی‌های گره‌ها را نیز تشخیص داده و مدیریت می‌کند تا یک سرویس محاسباتی توزیع‌شده‌ی مطمئنی را به‌صورت کارآمد فراهم نماید.

Hadoop حول سه مؤلفه‌ی زیر سازمان‌دهی می‌شود:

۱- یک سیستم‌فایل توزیع‌شده که داده‌ها و فایل‌ها را در میان گره‌های محاسباتی توزیع‌شده مدیریت می‌کند.

۲- چارچوب YARN[4] که منابع را در میان خوشه، همین‌طور زمان‌بندی وظایف بر روی گره‌های خوشه مدیریت می‌کند.

۳- سیستم MapReduce که امکان پردازش موازی داده‌ها را در میان گره‌های خوشه فراهم می‌کند.

Hadoop طراحی شده تا بر روی سیستم‌های لینوکس اجرا شود و برنامه‌های Hadoop می‌توانند با استفاده از چندین زبان برنامه‌نویسی، شامل زبان‌های اسکریپتی همچون PHP، Perl، و پایتون نوشته شوند. جاوا یک گزینه‌ی انتخابی مرسوم برای نوشتن برنامه‌های Hadoop می‌باشد، چرا که Hadoop دارای چندین کتابخانه‌ی جاوا است که MapReduce را پشتیبانی می‌کنند. اطلاعات بیشتر راجع به MapReduce و Hadoop را می‌توانید در https://hadoop.apache.org/docs/r1.2.1/mapred_tuturial.html و https://hadoop.apache.org پیدا کنید.

۱-۴-۱ چندبرنامگی و چندوظیفگی

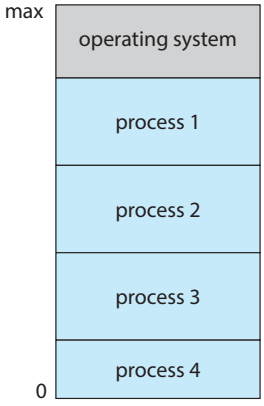
یکی از مهمترین وجوه سیستم‌عامل‌ها توانمندی اجرای برنامه‌های متعدد است، چرا که یک

1. Trap
2. System call
3. Big data
4. Yet Another Resource Negotiator (YARN)

برنامه‌ی منفرد در حالت کلی نمی‌تواند CPU و یا دستگاه‌های I/O را در تمامی زمان‌ها مشغول نگه بدارد. به‌علاوه، کاربران نوعاً *می‌خواهند* بیشتر از یک برنامه را در هر لحظه اجرا کنند. **چندبرنامگی**، بهره‌وری پردازنده را، در کنار جلب رضایت کاربران، با سازمان‌دهی برنامه‌ها افزایش می‌دهد، طوری که CPU همواره یک برنامه برای اجرا در اختیار دارد. در یک سیستم چندبرنامه‌ای، برنامه‌ی در حال اجرا، اصطلاحاً یک **پردازه** نامیده می‌شود.

ایده به این صورت بیان می‌شود: سیستم‌عامل چندین پردازه را همزمان در حافظه نگه می‌دارد (شکل ۱-۱۲). سیستم‌عامل یکی از این پردازه‌ها را برداشته و شروع می‌کند. در نهایت، این پردازه ممکن است برای انجام برخی وظایف، همچون تکمیل عملیات I/O، به ناچار منتظر شود. در یک سیستم فاقد چندبرنامگی، CPU بیکار خواهد نشست. در یک سیستم چندبرنامگی، سیستم‌عامل به سادگی به پردازه‌ی دیگری سوییچ کرده و آن را اجرا می‌کند. زمانی که لازم است *آن* پردازه هم منتظر شود، پردازنده به پردازه‌ی *دیگری* سوییچ کرده و الی آخر. سرانجام، پردازه‌ی اول، انتظارش به پایان رسیده و دوباره به CPU بر می‌گردد. تا زمانی که حداقل یک پردازه برای اجرا نیاز دارد، CPU هرگز بیکار نمی‌ماند.

این ایده در شرایط دیگر زندگی هم مرسوم است. برای مثال، یک وکیل فقط روی یک موکل کار نمی‌کند. در حالی‌که پرونده‌ای به انتظار حکم دادگاه به سر می‌برد یا نیاز به نامه‌نگاری دارد، وکیل می‌تواند روی پرونده‌ی دیگری کار کند. اگر وکیل به تعداد کافی موکل در اختیار داشته باشد هرگز به خاطر نبود کار بیکار نمی‌نشیند (وکلای بیکار، تمایل به سیاستمدار شدن دارند، لذا مشغول نگهداشتن و کلاً ارزش اجتماعی خاصی دارد).



شکل ۱-۱۲ چیدمان حافظه در یک سیستم چندبرنامگی.

چندوظیفگی[1] بسط منطقی چندبرنامگی است. در سیستم‌های چندوظیفگی، CPU چندین پردازه را با جابه‌جایی میان آنها اجرا می‌کند، این جابه‌جایی‌ها، چنان مکرر روی می‌دهند که کاربران یک **زمان پاسخ** سریعی را احساس می‌کنند. در نظر داشته باشید موقعی که یک پردازه

1. Multitasking

اجرا می‌شود، نوعاً برای یک زمان کوتاهی اجرا می‌شود پیش از این که خاتمه یابد و یا به انجام I/O نیاز داشته باشد. I/O ممکن است محاوره‌ای باشد؛ یعنی، خروجی در یک نمایشگر برای کاربر نمایش داده می‌شود، و ورودی از صفحه‌کلید کاربر، ماوس، یا صفحه‌نمایش لمسی می‌آید. از آنجایی که I/O محاوره‌ای نوعاً با «سرعت افراد» اجرا می‌شود، ممکن است زمان طولانی ببرد تا تکمیل شود. ورودی، برای مثال، برای مثال، ورودی ممکن است به‌واسطه‌ی سرعت تایپ کاربر محدود شود، تایپ هفت کاراکتر در ثانیه برای یک کاربر تند است، اما برای کامپیوترها بسیار کند است. سیستم‌عامل به جای این که پردازنده را در زمان انجام این ورودی محاوره‌ای معطل بگذارد، سریعاً CPU را به پردازه‌ی کاربر دیگری منتقل می‌کند.

داشتن چندین پردازه در حافظه به‌صورت هم‌زمان مستلزم شکلی از مدیریت حافظه است که در فصل ۹ و فصل ۱۰ پوشش می‌دهیم. به‌علاوه، اگر چندین پردازه هم‌زمان برای اجرا آمادگی داشته باشند، سیستم بایستی انتخاب کند که کدام پردازه در نوبت بعدی اجرا خواهد شد. اتخاذ این تصمیم، **زمان‌بندی CPU** می‌باشد که در فصل ۵ بحث می‌شود. در نهایت، اجرا همروند پردازه‌های متعدد مستلزم آن است که توانمندی آن‌ها در تأثیرگذاری روی همدیگر در تمامی مراحل سیستم‌عامل، اعم از زمان‌بندی پردازه، ذخیره‌سازی دیسک، و مدیریت حافظه محدود شود. ما این موضوعات را در کل این کتاب بحث می‌کنیم.

در یک سیستم چندوظیفگی، سیستم‌عامل بایستی زمان پاسخ مقبولی را تضمین نماید. یک روش رایج برای انجام این کار، **حافظه‌ی مجازی** است، تکنیکی که امکان اجرای یک پردازه‌ای را که به‌صورت کامل در داخل حافظه نیست فراهم می‌کند (فصل ۱۰). برتری اصلی این طرح در آن است که کاربران را قادر می‌سازد تا برنامه‌هایی را اجرا کنند که بزرگ‌تر از **حافظه‌ی فیزیکی** واقعی هستند. به‌علاوه، این موضوع، حافظه‌ی اصلی را به‌شکل یک آرایه‌ای از حافظه‌ی یک‌شکل بزرگ انتزاعی می‌سازد و با این کار، **حافظه‌ی منطقی** را که توسط کاربر دیده می‌شود از حافظه‌ی فیزیکی جدا می‌نماید. این شکل از مدیریت حافظه، برنامه‌نویسان را از نگرانی محدودیت‌های ذخیره‌سازی حافظه رها می‌سازد.

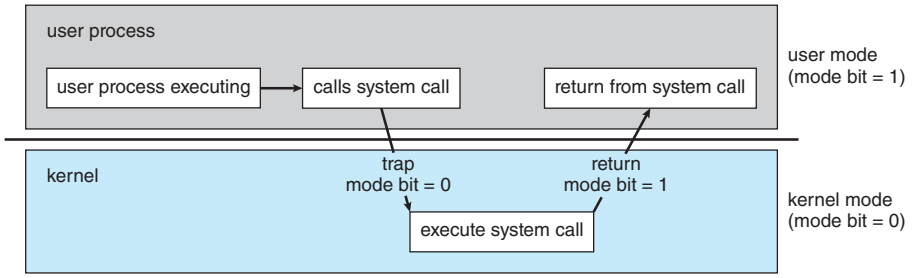
سیستم‌های چندبرنامگی و چندوظیفگی بایستی یک سیستم‌فایل نیز تدارک ببینند (فصل ۱۳، فصل ۱۴، و فصل ۱۵). سیستم‌فایل بر روی یک حافظه‌ی ثانویه مستقر می‌شود؛ از این رو، مدیریت ذخیره‌سازی بایستی فراهم شود (فصل ۱۱). به‌علاوه، یک سیستم بایستی منابع را از استفاده‌ی نامناسب محافظت کند (فصل ۱۷). برای تضمین اجرای مرتب، سیستم بایستی راهکارهایی نیز برای همگامی و ارتباط پردازه (فصل ۶ و فصل ۷) فراهم کند، و می‌تواند تضمین کند که پردازه‌ها در یک بن‌بست، به‌انتظار یکدیگر، تا ابد گرفتار نمی‌شوند (فصل ۸).

۱-۴-۲ عملیات مد دوگانه و چندمدی

از آنجایی که سیستم‌عامل و کاربران آن، منابع سخت‌افزاری و نرم‌افزاری سیستم کامپیوتری را به‌اشتراک می‌گیرند، یک سیستم‌عاملی که به‌درستی طراحی شده است بایستی تضمین نماید که یک برنامه‌ی نادرست (یا مخرب) نمی‌تواند باعث اجرای نادرست برنامه‌های دیگر—یا خود سیستم‌عامل—شود. ما به‌منظور تضمین اجرای درست سیستم بایستی قادر باشیم بین اجرای

کد سیستم‌عامل و کد تعریف‌شده‌ی کاربر تمایز قایل شویم. رویکردی که اکثر سیستم‌های کامپیوتری انتخاب می‌کنند تدارک پشتیبانی سخت‌افزاری است که امکان تمایز میان مدهای اجرایی مختلف را فراهم می‌کند.

حداقل، به دو **مد** عملیاتی مجزا نیاز داریم: **مد کاربر** و **مد هسته** (که **مد سوپروایزر**، **مد سیستم** یا **مد ممتاز**[1] هم نامیده می‌شود). یک بیت، به‌نام **بیت مد** به سخت‌افزار کامپیوتر افزوده می‌شود تا **مد** جاری را نشان بدهد: هسته (۰) یا کاربر (۱). با استفاده از بیت مد می‌توانیم وظیفه‌ای را که در جناح سیستم‌عامل اجرا می‌شود از وظیفه‌ی اجرایی طرف کاربر متمایز کنیم. زمانی که سیستم کامپیوتری در طرف برنامه‌ی کاربر اجرا می‌شود، سیستم در مد کاربر قرار دارد. با این وجود، لحظه‌ای که برنامه‌ی کاربر، سرویسی را از سیستم‌عامل (با فراخوانی سیستمی) درخواست می‌کند، سیستم بایستی برای انجام این درخواست از مد کاربر به مد هسته انتقال یابد. این روال در شکل ۱–۱۳ نشان داده می‌شود. چنان که خواهیم دید، این بهبود معماری برای بسیاری از وجوه دیگر عملیات سیستم نیز سودمند است.



شکل ۱–۱۳ انتقال از مد کاربر به مد هسته.

سخت‌افزار در زمان راه‌اندازی سیستم، کار را در مد هسته آغاز می‌کند. سپس، سیستم‌عامل بار شده و اجرای برنامه‌های کاربر را در مد کاربر شروع می‌کند. هر بار که تله یا وقفه اتفاق می‌افتد، سخت‌افزار از مد کاربر به مد هسته بر می‌گردد (یعنی، حالت بیت مد را به ۰ تغییر می‌دهد). لذا، هر موقعی که سیستم‌عامل کنترل کامپیوتر را به‌دست می‌گیرد، در مد هسته است. سیستم، همواره پیش از آنکه کنترل در اختیار برنامه‌ی کاربر قرار بگیرد، به مد کاربر بر می‌گردد (با تغییر بیت مد به ۱).

عملیات مد دوگانه ابزارهایی در اختیار ما می‌گذارد که بتوانیم سیستم‌عامل را از کاربران مخرب و کاربران مخرب را از همدیگر محافظت کنیم. این حفاظت را با مشخص‌کردن دستورالعمل‌هایی از ماشین که می‌توانند موجب خسارت شوند، تحت عنوان **دستورالعمل‌های ممتاز** انجام می‌دهیم. سخت‌افزار، اجرای دستورالعمل‌های ممتاز را تنها در مد هسته مجاز می‌داند. اگر اقدامی در مد کاربر برای اجرای دستورالعمل‌های ممتاز صورت بگیرد، سخت‌افزار دستورالعمل را اجرا نمی‌کند، بلکه آن را غیرقانونی قلمداد کرده و به سیستم‌عامل گزارش می‌دهد.

1. Privileged mode

دستورالعمل انتقال به مد هسته نیز نمونه‌ای از دستورالعمل‌های ممتاز است. مثال‌های دیگر عبارتند از: کنترل I/O، مدیریت تایمر و مدیریت وقفه. دستورالعمل‌های ممتاز فراوان دیگری نیز در کل کتاب مورد بحث قرار می‌گیرند.

مفهوم مد می‌تواند به بیش از دو مد تعمیم داده شود. برای مثال، پردازنده‌های اینتل دارای چهار **حلقه‌ی حفاظت**[1] مجزا هستند که حلقه‌ی ۰ مد هسته و حلقه‌ی ۳ مد کاربر است. (اگرچه حلقه‌های ۱ و ۲ می‌توانند برای سرویس‌های مختلف سیستم‌عامل استفاده شوند، آن‌ها در عمل به‌ندرت استفاده می‌شوند.) سیستم‌های ARMv8 هفت مد دارند. CPUهایی که از مجازی‌سازی پشتیبانی می‌کنند (بخش ۱۸-۱) در بیشتر موارد، یک مد مجزایی دارند که نشان می‌دهد چه موقع **مدیر ماشین مجازی** (**VMM**) در کنترل سیستم است. در این مد، امتیازات VMM از پردازه‌های کاربر بیشتر بوده، اما از هسته کمتر است. این سطح از امتیازات لازم است تا بتواند ماشین‌های مجازی را با تغییر وضعیت پردازنده، ایجاد و مدیریت کند.

اکنون می‌توانیم چرخه‌ی حیات اجرای دستورالعمل را در یک سیستم کامپیوتری ببینیم. کنترل آغازین در سیستم‌عامل مستقر است که دستورالعمل‌هایی در مد هسته اجرا می‌شوند. زمانی که کنترل به برنامه‌ی کاربر داده می‌شود، مد به مد کاربر برگردانده می‌شود. در نهایت، کنترل دوباره از طریق وقفه، تله یا فراخوانی سیستمی به سیستم‌عامل بر می‌گردد. اکثر سیستم‌عامل‌های کنونی—همچون مایکروسافت ویندوز، یونیکس، و لینوکس—از این امکان مد دوگانه بهره برده و حفاظت بیشتری برای سیستم‌عامل فراهم می‌کنند.

فراخوانی‌های سیستمی، ابزاری در اختیار برنامه‌ی کاربر قرار می‌دهند تا از سیستم‌عامل بخواهد کارهایی که برای سیستم‌عامل محفوظ هستند را از طرف برنامه‌ی کاربر انجام بدهد. فراخوانی سیستمی، بسته به کارکردی که پردازنده‌ی زیرین فراهم می‌کند به روش‌های مختلف می‌تواند احضار شود. فراخوانی سیستمی به هر شکلی که باشد روشی است که کاربر با استفاده از آن، انجام کاری را از سیستم‌عامل درخواست می‌کند. فراخوانی سیستمی معمولاً به شکل یک تله به مکان خاصی از بردار وقفه می‌باشد. این تله می‌تواند با دستورالعمل trap عمومی اجرا شود، اگرچه برخی سیستم‌ها یک دستورالعمل syscall ویژه هم برای احضار فراخوانی سیستمی دارند.

زمانی که یک فراخوانی سیستمی اجرا می‌شود، سخت‌افزار نوعاً با آن به عنوان یک وقفه‌ی نرم‌افزاری رفتار می‌کند. کنترل از طریق بردار وقفه به یک روال سرویس سیستم‌عامل منتقل شده و بیت مد به هسته تبدیل می‌شود. روال سرویس فراخوانی سیستمی، قسمتی از سیستم‌عامل است. هسته دستورالعمل صادرکننده‌ی وقفه را بررسی می‌کند تا تعیین کند، کدام یک از فراخوانی‌های سیستمی اتفاق افتاده است؛ در اینجا پارامتری وجود دارد که نشان می‌دهد، برنامه‌ی کاربر خواهان چه نوع سرویسی است. اطلاعات اضافه‌ای که برای درخواست مورد نیاز است می‌توانند به‌وسیله‌ی ثبات‌ها، از طریق پشته یا حافظه (با اشاره‌گرهایی به مکان‌های حافظه که به‌وسیله‌ی ثبات‌ها ارسال می‌شوند) ارسال شوند. هسته صحت و اعتبار پارامترها

1. Protection ring

را وارسی کرده، سپس درخواست را اجرا و کنترل را به دستورالعمل بعد از دستورالعمل فراخوانی سیستمی بر می‌گرداند. وقفه‌ها به صورت کامل‌تر در بخش ۲-۳ تشریح می‌شوند.

در صورتی که حفاظت سخت‌افزاری برقرار باشد، خطاهای نقض مدها را تشخیص می‌دهد. این خطاها طبق معمول توسط سیستم‌عامل رسیدگی می‌شوند. در هر صورت، اگر برنامه‌ی کاربر خطا داشته باشد—مثلاً اقدامی برای اجرای یک دستور غیر قانونی و یا دسترسی حافظه‌ای که در فضای آدرس کاربر نیست—در این صورت، سخت‌افزار به سیستم‌عامل تله صادر می‌کند. تله همانند یک وقفه، کنترل را از طریق بردار وقفه به سیستم‌عامل انتقال می‌دهد. زمانی که یک خطای برنامه اتفاق می‌افتد، سیستم‌عامل بایستی برنامه را به صورت غیرعادی[1] خاتمه بدهد. به این شرایط چنان رسیدگی می‌شود که گویی، خود کاربر، کد ختم غیرعادی را درخواست کرده است. پیغام خطای مناسبی صادر شده و امکان دارد رونوشتی از حافظه‌ی برنامه هم تولید شود. رونوشت حافظه معمولاً در یک فایل نوشته می‌شود تا کاربر یا برنامه‌نویس بتواند آن را بررسی و احتمالاً تصحیح کرده و برنامه را دوباره اجرا نماید.

۱-۴-۳ تایمر

بایستی تضمین کنیم، سیستم‌عامل کنترل پردازنده را در اختیار دارد. نمی‌توانیم به برنامه‌ی کاربر اجازه بدهیم که در حلقه‌ی نامتناهی بیفتد یا در فراخوانی سرویس‌های سیستم با خطا مواجه شده و هرگز کنترل را به سیستم‌عامل بر نگرداند. برای حصول این هدف می‌توانیم از **تایمر** استفاده کنیم. تایمر را می‌توان طوری تنظیم کرد تا بعد از سپری شدن دوره‌ی تناوب معینی، به کامپیوتر وقفه ارسال کند. تناوب زمانی می‌تواند ثابت (برای مثال، 1/60 ثانیه) یا متغیر (برای مثال، از ۱ میلی‌ثانیه تا ۱ ثانیه) باشد. به طور کلی، **تایمر متغیر** توسط یک ساعت با نرخ ثابت[2] و یک شمارنده پیاده‌سازی می‌شود. سیستم‌عامل شمارنده را تنظیم می‌کند. هر بار که ساعت تیک می‌خورد، شمارنده یکی کم می‌شود. لحظه‌ای که شمارنده به صفر می‌رسد، یک وقفه روی می‌دهد. برای نمونه، یک شمارنده‌ی ۱۰ بیتی با یک ساعت ۱ میلی‌ثانیه‌ای می‌تواند وقفه‌هایی در فواصل زمانی ۱ میلی‌ثانیه تا ۱۰۲۴ میلی‌ثانیه با گام‌های ۱ میلی‌ثانیه‌ای تولید کند.

پیش از این که کنترل به کاربر برگردد، سیستم‌عامل مطمئن می‌شود که تایمر جهت صدور وقفه تنظیم شده است. اگر تایمر وقفه صادر کند، کنترل به طور خودکار به سیستم‌عامل منتقل می‌شود، که امکان دارد وقفه را یک خطای مخرب تلقی کند یا به برنامه زمان بیشتری بدهد. روشن است که دستورالعمل‌هایی که محتوای تایمر را تغییر می‌دهند ممتاز خواهند بود.

تایمرهای لینوکس

در سیستم‌های لینوکس، پارامتر پیکربندی هسته، HZ، فرکانس وقفه‌های تایمر را مشخص می‌کند. مقدار ۲۵۰ برای HZ به این معنی است که تایمر ۲۵۰ وقفه در هر ثانیه، یا یک وقفه در هر ۴ میلی‌ثانیه

1. Abnormally
2. Fixed-rate